

Pseudocode Solution 1:

Input:

- N = 1000: The size of the array (for both arrays A and B).
- Array A: An array of integers of size N, filled with random numbers between 1 and 50.

Output:

- Array B: Array B is calculated based on direct multiplication of elements from array A.
- Execution Time: Time taken to calculate array B using direct multiplication.
- Printed Array B: Display the values of array B after calculation.

// High-Level Algorithm Start:

MODULE SOLUTIONS

BEGIN MAIN

 // Function prototypes

 DECLARE FUNCTION generateRandomArray1(Array, Size)

 DECLARE FUNCTION calculateArrayB_Direct1(A, B, Size)

 DECLARE FUNCTION printArray1(Array, Size)

 CONSTANT N = 1000

 DECLARE A[N], B[N]

 // Initialize random number generator

 CALL srand(time(0))

 // Generate random values for array A

 CALL generateRandomArray1(A, N)

 // Measure time for Solution 1 (Direct Calculation)

 DECLARE start1 = clock()

 CALL calculateArrayB_Direct1(A, B, N)

```

        DECLARE end1 = clock()

        // Calculate time taken
        DECLARE time_taken1 = (end1 - start1) /
CLOCKS_PER_SEC

        PRINT "Solution 1 (Direct Multiplication) Time: ",
time_taken1

        // Print array B
        PRINT "Array B (Solution 1 - Direct Multiplication):"
        CALL printArray1(B, N)

END MODULE

END MAIN

// High-Level Algorithm End:

// Low-Level Algorithm Start:
Step 1: Refinement
FUNCTION generateRandomArray1(Array, Size)
    FOR i FROM 0 TO Size-1 DO
        Array[i] = Random number between 1 and 50
    END FOR
END FUNCTION

Step 2: Refinement
FUNCTION calculateArrayB_Direct1(A, B, Size)
    FOR i FROM 0 TO Size-1 DO
        DECLARE product = 1
        FOR j FROM 0 TO i DO
            product = product * A[j] // Multiply A[0] to
A[i] to get the product

```

```

        END FOR
        B[i] = product // Assign the product to B[i]
    END FOR
END FUNCTION

```

Step 3: Refinement

```

FUNCTION printArray1(Array, Size)
    FOR i FROM 0 TO Size-1 DO
        PRINT Array[i]
    END FOR

    PRINT newline // To move to the next line after
printing all elements
END FUNCTION

```

// Low-Level Algorithm End:

Pseudocode Solution 2:

// Input:

- N = 1000 (Size of array A with random integer values)
- Integer array A[N] (generated with random values between 1 and 50)

// Output:

- Integer array B[N] (cumulative product up to each index i in A)
- Execution time for the optimized calculation of array B

// High-Level Algorithm:

```

BEGIN MAIN

    // Function prototypes
    DECLARE FUNCTION generateRandomArray2(array, size)
    DECLARE FUNCTION calculateArrayB_Optimized2(A, B,
size)
    DECLARE FUNCTION printArray2(array, size)


    CONSTANT SIZE = 1000
    DECLARE Integer Array A[SIZE]
    DECLARE Integer Array B[SIZE]


    // Generate random values for array A
    CALL generateRandomArray2(A, SIZE)


    // Measure start time for execution time calculation
    DECLARE Float start_time = CURRENT_TIME()


    // Calculate array B using cumulative product
    approach
    CALL calculateArrayB_Optimized2(A, B, SIZE)


    // Measure end time and calculate execution time
    DECLARE Float end_time = CURRENT_TIME()
    DECLARE Float execution_time = end_time - start_time


    // Display results
    PRINT "Execution time for Solution 2 (Optimized): ",
execution_time, " seconds"
    PRINT "Array B (Optimized):"
    CALL printArray2(B, SIZE)
END MAIN


// High-Level Algorithm End

```

```
// Low-Level Algorithm Start
```

```
Step 1 Refinement
```

```
FUNCTION generateRandomArray2(array, size)
```

```
    FOR i FROM 0 TO size - 1 DO
```

```
        SET array[i] = RANDOM_NUMBER() MOD 50 + 1  //  
Generate random integer between 1 and 50
```

```
    END FOR
```

```
END FUNCTION
```

```
Step 2 Refinement
```

```
FUNCTION calculateArrayB_Optimized2(A, B, size)
```

```
    SET B[0] = A[0]  // Initialize first element of B
```

```
    FOR i FROM 1 TO size - 1 DO
```

```
        SET B[i] = B[i - 1] * A[i]  // Cumulative product  
of A up to index i
```

```
    END FOR
```

```
END FUNCTION
```

```
Step 3 Refinement
```

```
FUNCTION printArray2(array, size)
```

```
    FOR i FROM 0 TO size - 1 DO
```

```
        PRINT array[i], " "
```

```
    END FOR
```

```
    PRINT NEW_LINE
```

```
END FUNCTION
```

```
// Low-Level Algorithm End
```

Solution 2:

Solution 2 is faster than Solution 1 because it avoids recalculating products by using cumulative multiplication, resulting in $O(N)$ complexity instead of $O(N^2)$. This optimized approach reduces the number of operations significantly, especially for large arrays, and leads to faster execution times.